ASSIGNMENT 3

Database Design Analysis

Event Ticketing and Venue Seating System

Name	Aishani Dutta
Registration Number	2511021240020
Course / Section	BCA SEM 3 2025
Date	23 October 2026

About the design

This system lets a customer choose an event, select seats and make a payment. It has seven tables: Users, Venues, Seats, Events, Bookings, Booking_Seats and Payments. A booking can contain more than one seat. Payments are stored separately so that a failed attempt and a later retry can both be recorded.

Why separate tables are needed

If all details were written in one booking register, the customer name, event title and venue address would be repeated for every seat. For example, booking 201 contains two seats for Acoustic Evening. The customer and event details should be stored once, while the two seat selections should have separate rows.

Booking	Customer	Event	Seat	Price (Rs.)
201	Ananya	Acoustic Evening	1	500
201	Ananya	Acoustic Evening	2	500

In the final design, Users stores Ananya once, Events stores Acoustic Evening once, and Booking_Seats contains the two seat rows. Bookings connects them using identifiers.

Problems with a single register

Problem	Example	Improvement
Insertion anomaly	A new venue cannot be entered unless a booking is also created.	Store the venue in Venues before adding events or bookings.
Update anomaly	Changing a venue address requires updating every booking row that repeats it. Some rows may retain the old address.	Update one row in Venues.
Deletion anomaly	Deleting the only booking for an event may also remove the only stored event details.	Keep the event in Events independently of its bookings.

The problem is that the register mixes details about customers, events, seats and payments. Each of these changes for a different reason, so they should not all depend on one booking row.

Functional dependencies

A functional dependency means that one attribute determines another. For example, user_id determines the name and email of that user.

Table	Determines	Dependent attributes
Users	user_id	name, phone, email, password_hash, created_at
Venues	venue_id	name, address, city, total_capacity
Seats	seat_id	venue_id, section, row_label, seat_number, seat_type
Events	event_id	venue_id, title, category, event_date, start_time, description
Bookings	booking_id	user_id, event_id, booking_time, total_amount, booking_status
Booking_Seats	booking_seat_id	booking_id, seat_id, event_id, price, ticket_code
Payments	payment_id	booking_id, amount, payment_method, payment_status, paid_at

Email and phone are also unique identifiers for Users in this design. For Seats, the combination of venue_id, section, row_label and seat_number identifies one seat. A ticket_code identifies one Booking_Seats row. The pair (event_id, seat_id) must also be unique so that one seat cannot be allocated twice for the same event.

Remaining redundancy in this design

Booking_Seats contains event_id even though booking_id already determines the event through Bookings. This is a real repeated dependency: booking_seat_id determines booking_id, which determines event_id. Because booking_id can occur in several seat rows, it is not a key of Booking_Seats. This part of the design is therefore not fully in third normal form.

The repeated event_id is kept here to enforce UNIQUE(event_id, seat_id). A combined Foreign Key on (booking_id, event_id) ensures that the event in the ticket row matches the event in the booking. This makes the repeated value controlled instead of leaving it unchecked.

Bookings.total_amount also repeats a value that can be calculated by adding seat prices. It must be updated in the same transaction as seat changes. Venues.total_capacity should match the complete seat map; the sample data uses six seats for each small venue.

Normalization and improvements

First normal form: Each seat selection is stored in its own Booking_Seats row. Seat numbers are not stored as a comma-separated list inside Bookings.

Second normal form: Details such as the venue address and user email are stored with their own identifiers. In a flat register keyed by booking_id and seat_id, booking_time would depend only on booking_id, which would be a partial dependency.

Third normal form: Separating Users, Venues and Events removes chains such as booking_id to user_id to email. The event_id repetition in Booking_Seats remains for the seat-allocation constraint described above.

Change	Purpose
Keep user, event and venue details separate	Avoid repeated names, addresses and contact details.
Use UNIQUE(event_id, seat_id)	Prevent two allocations of the same seat for one event.
Use a combined Foreign Key for the booking and event	Prevent a ticket from referring to a different event than its booking.
Check that the seat and event use the same venue	Prevent assigning a seat from another location.
Calculate availability from Booking_Seats	Avoid a separate available flag that could become incorrect.
Check totals when seats are added or removed	Keep total_amount equal to the sum of current seat prices.

In this version, Booking_Seats stores current allocations. Pending bookings can hold seats. Cancelling a booking must remove its seat allocations and update its total in one transaction. Simply changing the status does not release a seat. Payments and the booking record can remain as history; a full cancelled-ticket archive is outside this design.